Document number: P2855R1
Audience: LEWG

Ville Voutilainen
2024-02-22

Member customization points for Senders and Receivers

Abstract

There have been various suggestions that Senders and Receivers need a new language feature for customization points, to avoid the complexity of ADL tag_invoke.

This paper makes the case that C++ already has such a language facility, and it works just fine for the purposes of Senders and Receivers.

That language facility is member functions.

In a nutshell, the approach in this paper is relatively straightforward; for all non-query customization points, ADL tag_invoke overloads become member functions. Query customization points become query member functions that take the query tag as an argument.

This is because non-queries don't need to forward calls to customization points, but it's useful for queries to be able to forward queries.

In order to be able to write perfect-forwarding function templates that work both for lvalues and rvalues, we use deduced this. When there is no need to write a single function for both lvalues and rvalues, a traditional non-static member function will do.

The overall highest-priority goal of this proposal is "No ADL, *anywhere*"

Quick examples

A tag_invoke customization point for start

```
friend void tag_invoke(std::execution::start_t, recv_op& self) noexcept
```

becomes

```
void start() & noexcept
```

A perfect-forwarding connect

```
template <__decays_to<__t> _Self, receiver _Receiver>
    requires sender_to<__copy_cvref_t<_Self, _Sender>, __receiver<_Receiver>>
friend auto tag_invoke(std::execution::connect_t, _Self&& __self, _Receiver __rcvr)
```

becomes

```
template <__decays_to<__t> _Self, receiver _Receiver>
    requires sender_to<__copy_cvref_t<_Self, _Sender>, __receiver<_Receiver>>
auto connect(this _Self&& __self, _Receiver __rcvr)
```

The call

```
tag_invoke(std::execution::connect, std::forward<Snd>(s), r);
```

becomes

```
std::forward<Snd>(s).connect(r);
```

A query

```
friend in_place_stop_token tag_invoke(std::execution::get_stop_token_t, const __t& __self) noexcept
```

becomes

```
in_place_stop_token query(std::execution::get_stop_token_t) const noexcept
```

A note on what changed there from R0

After an LEWG discussion where it was suggested that tag parameters/arguments are untoward, and a very helpful suggestion that if we have wrappers anyway, we can use nested types instead, various people discussing this came to the conclusion that that feedback is right - we don't need the tags, except for query. We can add member typedef opt-ins to operation states, like we already have in receivers, and then we don't need those tag parameters/arguments.

Furthermore, we don't need to name the query function a "tag_query". It's a query, it takes a tag, but that tag-taking doesn't need to go into the name. It's a member function. If you manage to mix such functions in a wrapper class, don't do it. Don't multi-inherit things into your sender wrapper, don't multi-inherit a sender wrapper and something else. Or if you do, use whatever usual techniques to disambiguate declarations and calls, but mostly just don't do it.

What does this buy us?

First of all, two things, both rather major:

1. NO ADL.
2. ..and that makes defining customization points *much* simpler.

A bit of elaboration on the second point: consider that earlier query of get_stop_token in tag_invoke form. It's an example of that query for the when_all algorithm. But what needs to be done is that both that query (which is a hidden friend) and the when_all_t function object type are in a detail-namespace, and then outside that namespace, in namespace std::execution, the type is brought into scope with a using-declaration, and the actual function object is defined.

Roughly like this:

```
namespace you_will_have_trouble_coming_up_with_a_name_for_it {
  template <class Snd, class Recv, class Fn>
  struct my_then_operation {
    opstate op;
    struct t {
      friend void tag_invoke(start_t, t& self) noexcept {
        start(self.op_);
      }
    };
  };
  // ADL-protected internal senders and receivers omitted
  struct my_then_t {
    template <sender Snd, class Fn> // proper constraints omitted
    sender auto operator()(Snd&& sndr, Fn&& fn) const {
      // the actual implementation omitted
    }
  };
}
using you_will_have_trouble_coming_up_with_a_name_for_it::my_then_t;
constexpr my_then_t my_then{};
```

This has the effect of keeping the overload set small, when each and every type and its customizations are meticulously defined that way. Build times are decent, the sizes of overload sets are nicely controlled and are small, diagnostics for incorrect calls are hopefully fairly okay.

But that's not all there is to it. Generic code that uses such things should wrap its template parameters into utilities that prevent ADL via template parameters. You might see something like this gem:

```
// For hiding a template type parameter from ADL
template <class _Ty>
struct _X {
  using __t = struct _T {
    using __t = _Ty;
  };
};
template <class _Ty>
using __x = __t<_X<_Ty>>;
```

and then use it like this:

```
using make_stream_env_t = stream_env<stdexec::__x<BaseEnv>>;
```

With member customization points, you don't need any such acrobatics. The customization points are members. You define a customization point as a member function, and you can just put your type directly into whichever namespace you want (some might even use the global namespace), and you don't need to use nested detail namespaces. Then you call `foo.connect(std::execution::connect, receiver);` and you don't have to do the no-ADL wrapping in your template parameters either.

In other words, the benefits of avoiding ADL for the implementation include

- no need to wrap template parameters, to avoid making their associated namespaces be considered for ADL
- no need to wrap a sender algorithm's internal operation states, senders, and receivers into nested namespaces, to limit the searched scopes when ADL would be applied on such a type
- no need to wrap a sender algorithm into a nested namespaces and do a using-declaration in an outer one, again to limit the seached scopes when ADL would be applied on something else, wishing to avoid finding the functions in the namespace of the algorithm.

Some of those are fairly traditional ADL-taming techniques, some may be recent realizations. None of them are necessary when members are used, none. This should greatly simplify the implementation. The benefits for the users are mostly the same, they don't need to apply any of those techniques, not for their custom schedulers, not for their custom senders, not for their algorithm customizations, not for anything.

The definition of customization points is much simpler, to a ridiculous extent. Using them is simpler; it's a member call, everybody knows what that does, and many people know what scopes that looks in, and a decent amount of people appreciate the many scopes it *doesn't* look in.

Composition and reuse and wrapping of customization points becomes much easier, because it's just.. ..good old OOP, if you want to look at it that way. We're not introducing a new language facility for which you need to figure out how to express various function compositions and such, the techniques and patterns are decades old, and work here as they always worked.

What are its downsides compared to a new language facility?

Well, we don't do anything for users who for some reason _have_ to use ADL customization points. But the reason for going for this approach is that we decouple Senders and Receivers from an unknown quantity, and avoid many or even most of the problems of using ADL customization points.

Other than that, I'm not sure such downsides exist.

A common concern with using wrappers is that they don't work if you have existing APIs that use the wrappees - introducing wrappers into such situations just doesn't work, because they simply aren't the same type, and can't be made the same type. And a further problem is having to deal with both the wrappers and wrappees as concrete types, and figuring out when to use which, and possibly having to duplicate code to deal with both.

The saving grace with Senders and Receivers is that they are wrapped everywhere all the time. Algorithms wrap senders, the wrapped senders wrap their receivers, and resulting operation states. This wrapping nests pretty much infinitely.

For cases where you need to use a concrete sender, it's probably type-erased, rather than being a use of a concrete target sender.

Implementation experience

A very partial work-in-progress implementation exists as a branch of the reference implementation of P2300, at https://github.com/villevoutilainen/wg21_p2300_std_execution/tree/P2855_member_customization_points.

The implementation has the beginnings of a change from ADL tag_invoke overloads to non-static member functions and member functions using deduced this. It's rather rudimentary, and very incomplete, only covering operation states at this point.

Some additional considerations

Access

It's possible to make customization point members private, and have them usable by the framework, by befriending the entry point (e.g. std::execution::connect, in a member connect(std::execution::connect_t)). It's perhaps ostensibly rare to need to do that, considering that it's somewhat unlikely that a sender wrapper or an operation state wrapper that provides the customization point would have oodles of other functionality. Nevertheless, we have made that possible in the prototype implementation, so we could do the same in the standard. This seems like an improvement over the ADL customization points. With them, anyone can do the ADL call, the access of a hidden friend doesn't matter.

Interface pollution

It's sometimes plausible that a class with a member customization point inherits another class that provides the same customization point, and it's not an override of a virtual function. In such situations, the traditional technique works, bring in the base customization point via a using-declaration, which silences possible hiding warnings and also create an overload set. The expectation is that the situation and the technique are sufficiently well-known, since it's old-skool.

Wording

General note: the goal here is to replace tag_invokes with member functions, and remove all ADL-mitigating techniques; there are other changes I deemed necessary at least for this presentation: none of the CPOs are meant to be called unqualified after this paper's change(s). They are not customizable as such, in and of themselves, despite being CPOs. They are entry points. The entry points are called qualified (and in some cases have to be; you can't just call a foo.connect() on any coroutine result type, but you can call std::execution::connect() on it.), and they are customized by the mechanism depicted in P2999, if the thing customized is an algorithm, or by writing member functions, if the thing customized is not really a customization but rather an opt-in.

But note, though, that once the adoption of this paper's approach is done, we don't *have* to qualify *anything* in this specification, because all calls to namespace-scope functions are *as-if* qualified, and the rest is member calls.

Additionally, it might be tempting to remove the function objects set_value, set_error and set_stopped completely, but there are things that use them as generic function objects (see *just-sender* below), so that ability is left as-is.

Due to not using ADL, 16.4.6.17 Class template-heads can be removed, as it's an ADL-mitigating technique that isn't necessary when member functions are used for everything.

In [functional.syn], strike tag_invocable, nothrow_tag_invocable, tag_invoke_result, and tag_invoke:

```
~~// [func.tag_invoke], tag_invoke~~
~~namespace tag_invoke { // exposition only~~
~~  void tag_invoke();~~

~~  template<class Tag, class... Args>~~
~~    concept tag_invocable =~~
~~      requires (Tag&& tag, Args&&... args) {~~
~~        tag_invoke(std::forward<Tag>(tag), std::forward<Args>(args)...);~~
~~      };~~

~~  template<class Tag, class... Args>~~
~~    concept nothrow_tag_invocable =~~
~~      tag_invocable<Tag, Args...> &&~~
~~      requires (Tag&& tag, Args&&... args) {~~
~~        { tag_invoke(std::forward<Tag>(tag), std::forward<Args>(args)...) } noexcept;~~
~~      };~~

~~  template<class Tag, class... Args>~~
~~    using tag_invoke_result_t =~~
~~      decltype(tag_invoke(declval<Tag>(), declval<Args>()...));~~

~~  template<class Tag, class... Args>~~
~~    struct tag_invoke_result<Tag, Args...> {~~
~~      using type =~~
~~        tag_invoke_result_t<Tag, Args...>; // present if and only if tag_invocable<Tag, Args...> is true~~
~~    };~~

~~  struct tag; // exposition only~~
~~}~~
~~inline constexpr tag_invoke::tag tag_invoke {};~~
~~using tag_invoke::tag_invocable;~~
~~using tag_invoke::nothrow_tag_invocable;~~
~~using tag_invoke::tag_invoke_result_t;~~
~~using tag_invoke::tag_invoke_result;~~

template<auto& Tag>
  using tag_t = decay_t<decltype(Tag)>;
```

Remove [func.tag_invoke]

In [exec.general]/p4.1, replace the specification of the exposition-only *mandate-nothrow-call* with the following:

1. For a subexpression *expr*, let *MANDATE-NOTHROW(expr)* be expression-equivalent to *expr*,

   Mandates: `noexcept(`*expr*`)` is `true`.

In [exec.syn], remove ADL-protecting nested namespaces:

```
~~namespace queries { // exposition only~~
    struct forwarding_query_t;
    struct get_allocator_t;
    struct get_stop_token_t;
  ~~}~~
~~using queries::forwarding_query_t;~~
~~using queries::get_allocator_t;~~
~~using queries::get_stop_token_t;~~

namespace std::execution {
  // [exec.queries], queries
  enum class forward_progress_guarantee;
  ~~namespace queries { // exposition only~~
    struct get_domain_t;
    struct get_scheduler_t;
    struct get_delegatee_scheduler_t;
    struct get_forward_progress_guarantee_t;
    template<class CPO>
      struct get_completion_scheduler_t;
  ~~}~~
~~using queries::get_domain_t;~~
~~using queries::get_scheduler_t;~~
~~using queries::get_delegatee_scheduler_t;~~
~~using queries::get_forward_progress_guarantee_t;~~
~~using queries::get_completion_scheduler_t;~~
  inline constexpr get_domain_t get_domain{};
  inline constexpr get_scheduler_t get_scheduler{};
  inline constexpr get_delegatee_scheduler_t get_delegatee_scheduler{};
  inline constexpr get_forward_progress_guarantee_t get_forward_progress_guarantee{};
  template<class CPO>
    inline constexpr get_completion_scheduler_t get_completion_scheduler{};

  ~~namespace exec-envs { // exposition only~~
    struct empty_env {};
    struct get_env_t;
  ~~}~~
~~using exec-envs::empty_env;~~
~~using exec-envs::get_env_t;~~

  // [exec.domain.default], domains
  struct default_domain;

  // [exec.sched], schedulers
  struct scheduler_t {};

  template<class Sch>
    concept scheduler = see below;

//...

  ~~namespace receivers { // exposition only~~
    struct set_value_t;
    struct set_error_t;
    struct set_stopped_t;
  ~~}~~
~~using receivers::set_value_t;~~
~~using receivers::set_error_t;~~
~~using receivers::set_stopped_t;~~

// ...

  ~~namespace op-state { // exposition only~~
    struct start_t;
  ~~}~~
~~using op-state::start_t;~~
  struct operation_state_t {};

// ...

  ~~namespace completion-signatures { // exposition only~~
    struct get_completion_signatures_t;
  ~~}~~
~~using completion-signatures::get_completion_signatures_t;~~

// ...

  ~~namespace senders-connect { // exposition only~~
    struct connect_t;
  ~~}~~
~~using senders-connect::connect_t;~~

// ...

  ~~namespace senders-factories { // exposition only~~
    struct just_t;
    struct just_error_t;
    struct just_stopped_t;
    struct schedule_t;
  ~~}~~

  ~~using senders-factories::just_t;~~
  ~~using senders-factories::just_error_t;~~
  ~~using senders-factories::just_stopped_t;~~
  ~~using senders-factories::schedule_t;~~
  inline constexpr just_t just{};
```

```
  inline constexpr just_error_t just_error{};
  inline constexpr just_stopped_t just_stopped{};
  inline constexpr schedule_t schedule{};
  inline constexpr unspecified read{};

// ...

  namespace sender-adaptor-closure { // exposition only
    template<class-type D>
      struct sender_adaptor_closure { };
  }
  using sender-adaptor-closure::sender_adaptor_closure;

  namespace sender-adaptors { // exposition only
    struct on_t;
    struct transfer_t;
    struct schedule_from_t;
    struct then_t;
    struct upon_error_t;
    struct upon_stopped_t;
    struct let_value_t;
    struct let_error_t;
    struct let_stopped_t;
    struct bulk_t;
    struct split_t;
    struct when_all_t;
    struct when_all_with_variant_t;
    struct into_variant_t;
    struct stopped_as_optional_t;
    struct stopped_as_error_t;
    struct ensure_started_t;
  }
  using sender-adaptors::on_t;
  using sender-adaptors::transfer_t;
  using sender-adaptors::schedule_from_t;
  using sender-adaptors::then_t;
  using sender-adaptors::upon_error_t;
  using sender-adaptors::upon_stopped_t;
  using sender-adaptors::let_value_t;
  using sender-adaptors::let_error_t;
  using sender-adaptors::let_stopped_t;
  using sender-adaptors::bulk_t;
  using sender-adaptors::split_t;
  using sender-adaptors::when_all_t;
  using sender-adaptors::when_all_with_variant_t;
  using sender-adaptors::into_variant_t;
  using sender-adaptors::stopped_as_optional_t;
  using sender-adaptors::stopped_as_error_t;
  using sender-adaptors::ensure_started_t;

// ...

  namespace sender-consumers { // exposition only
    struct start_detached_t;
  }
  using sender-consumers::start_detached_t;

// ...
}

namespace std::this_thread {
  // [exec.queries], queries
  namespace queries { // exposition only
    struct execute_may_block_caller_t;
  }
  using queries::execute_may_block_caller_t;
  inline constexpr execute_may_block_caller_t execute_may_block_caller{};

  namespace this-thread { // exposition only
    struct sync-wait-env; // exposition only
    template<class S>
        requires sender_in<S, sync-wait-env>
      using sync-wait-type = see below; // exposition only
    template<class S>
      using sync-wait-with-variant-type = see below; // exposition only

    struct sync_wait_t;
    struct sync_wait_with_variant_t;
  }
  using this-thread::sync_wait_t;
  using this-thread::sync_wait_with_variant_t;
}

namespace std::execution {
  // [exec.execute], one-way execution
  namespace execute { // exposition only
    struct execute_t;
  }
  using execute::execute_t;
  inline constexpr execute_t execute{};

  // [exec.as.awaitable]
  namespace coro-utils { // exposition only
    struct as_awaitable_t;
  }
  using coro-utils::as_awaitable_t;

  // [exec.with.awaitable.senders]
```

```
  template<class-type Promise>
    struct with_awaitable_senders;
}
```

In [exec.get.env]/1, edit as follows:

> execution::get_env is a customization point object. For some subexpression o of type O, execution::get_env(o) is expression-equivalent t
> ~~tag_invoke(std::get_env,~~ const_cast<const O&>(o).get_env() if that expression is well-formed.

In [exec.fwd.env]/2.1, edit the expression form:

> ~~*mandate-nothrow-call*(tag_invoke, std::forwarding_query, q)~~*MANDATE-NOTHROW*(q.query(std::forwarding_query)) if that expression is well-for

In [exec.get.allocator]/2, edit as follows:

> The name std::get_allocator denotes a query object. For some subexpression r, std::get_allocator(r) is expression-equivalent to
> ~~*mandate-nothrow-call*(tag_invoke, std::get_allocator, as_const(r))~~*MANDATE-NOTHROW*(as_const(r).query(std::get_allocator)).

In [exec.get.stop.token]/2, edit as follows:

> The name std::get_stop_token denotes a query object. For some subexpression r, std::get_stop_token(r) is expression-equivalent to:
> ~~*mandate-nothrow-call*(tag_invoke, std::get_stop_token, as_const(r))~~*MANDATE-NOTHROW*(as_const(r).query(std::get_stop_token)),
> if this expression is well-formed.

In [exec.get.scheduler]/2, edit as follows:

> The name execution::get_scheduler denotes a query object. For some subexpression r, execution::get_scheduler(r) is expression-equivalent
> ~~*mandate-nothrow-call*(tag_invoke, get_scheduler, as_const(r))~~*MANDATE-NOTHROW*(as_const(r).query(execution::get_scheduler)).

In [exec.get.scheduler]/4, edit as follows:

> execution::get_scheduler() (with no arguments) is expression-equivalent to execution::read(execution::get_scheduler)

In [exec.get.delegatee.scheduler]/2, edit as follows:

> The name execution::get_delegatee_scheduler denotes a query object. For some subexpression r, execution::get_delegatee_scheduler(r) is e
> ~~*mandate-nothrow-call*(tag_invoke, get_delegatee_scheduler, as_const(r))~~*MANDATE-NOTHROW*(as_const(r).query(execution::get_delegatee_schedul

In [exec.get.forward.progress.guarantee]/2, edit as follows:

> The name execution::get_forward_progress_guarantee denotes a query object. For some subexpression s, let S be decltype((s)).
> If S does not satisfy scheduler, get_forward_progress_guarantee is ill-formed.
> Otherwise, execution::get_forward_progress_guarantee(s) is expression-equivalent to:
>
> - ~~*mandate-nothrow-call*(tag_invoke, get_forward_progress_guarantee, as_const(s))~~
>   *MANDATE-NOTHROW*(as_const(s).query(execution::get_forward_progress_guarantee)), if this expression is well-formed.

In [exec.execute.may.block.caller]/2.1, edit the expression form:

> ~~*mandate-nothrow-call*(tag_invoke, this_thread::execute_may_block_caller, as_const(s))~~
> *MANDATE-NOTHROW*(as_const(s).query(this_thread::execute_may_block_caller)), if this expression is well-formed.

In [exec.completion.scheduler]]/2, edit as follows:

> The name execution::get_completion_scheduler denotes a query object template. For some subexpression q, let Q be decltype((q)).
> If the template argument Tag in get_completion_scheduler<Tag>(q) is not one of set_value_t, set_error_t, or set_stopped_t,
> get_completion_scheduler<Tag>(q) is ill-formed. Otherwise, execution::get_completion_scheduler<Tag>(q) is expression-equivalent to
> ~~*mandate-nothrow-call*(tag_invoke, get_completion_scheduler<Tag>, as_const(q))~~
> *MANDATE-NOTHROW*(as_const(q).query(execution::get_completion_scheduler<Tag>)) if this expression is well-formed.

In [exec.sched]/1, edit as follows:

```
template<class Sch>
  inline constexpr bool enable-scheduler = // exposition only
    requires {
      requires derived_from<typename Sch::scheduler_concept, scheduler_t>;
    };

template<class Sch>
  concept scheduler =
    enable-scheduler<remove_cvref_t<Sch>> &&
    queryable<Sch> &&
    requires(Sch&& sch, const get_completion_scheduler_t<set_value_t> tag) {
      { schedule(std::forward<Sch>(sch)) } -> sender;
      { tag_invoke(tag, std::get_env(
        execution::get_env(execution::schedule(std::forward<Sch>(sch))).query(tag)) }
          -> same_as<remove_cvref_t<Sch>>;
    } &&
    equality_comparable<remove_cvref_t<Sch>> &&
    copy_constructible<remove_cvref_t<Sch>>;
```

In [exec.recv.concepts]/1, edit as follows:

```
template<class Rcvr>
  inline constexpr bool enable_receiverenable-receiver = // exposition only
    requires {
      requires derived_from<typename Rcvr::receiver_concept, receiver_t>;
    };

template<class Rcvr>
  concept receiver =
    enable_receiverenable-receiver<remove_cvref_t<Rcvr>> &&
```

```
    requires(const remove_cvref_t<Rcvr>& rcvr) {
      { execution::get_env(rcvr) } -> queryable;
    } &&
    move_constructible<remove_cvref_t<Rcvr>> &&  // rvalues are movable, and
    constructible_from<remove_cvref_t<Rcvr>, Rcvr>; // lvalues are copyable
```

Strike [exec.recv.concepts]/2:

~~Remarks: Pursuant to [namespace.std], users can specialize enable_receiver to true for cv-unqualified program-defined types that model receiver, and false for types that do not. Such specializations shall be usable in constant expressions ([expr.const]) and ha~~

In [exec.set.value]/1, edit as follows:

execution::set_value is a value completion function ([async.ops]). Its associated completion tag is execution::set_value_t.
The expression execution::set_value(R, Vs...) for some subexpression R and pack of subexpressions Vs is ill-formed if R is an lvalue or
Otherwise, it is expression-equivalent to ~~*mandate-nothrow-call*(tag_invoke, set_value, R, Vs...)~~*MANDATE-NOTHROW*(R.set_value(Vs...)).

In [exec.set.error]/1, edit as follows:

execution::set_error is an error completion function. Its associated completion tag is execution::set_error_t.
The expression execution::set_error(R, E) for some subexpressions R and E is ill-formed if R is an lvalue or a const rvalue.
Otherwise, it is expression-equivalent to ~~*mandate-nothrow-call*(tag_invoke, set_error, R, E)~~*MANDATE-NOTHROW*(R.set_error(E)).

In [exec.set.stopped]/1, edit as follows:

execution::set_stopped is a stopped completion function. Its associated completion tag is execution::set_stopped_t.
The expression execution::set_stopped(R) for some subexpression R is ill-formed if R is an lvalue or a const rvalue.
Otherwise, it is expression-equivalent to ~~*mandate-nothrow-call*(tag_invoke, set_stopped, R)~~*MANDATE-NOTHROW*(R.set_stopped()).

In [exec.opstate]/1, edit as follows:

```
template<class O>
  inline constexpr bool enable-operation-state = // exposition only
    requires {
      requires derived_from<typename O::operation_state_concept, operation_state_t>;
    };

template<class O>
  concept operation_state =
    enable-operation-state<O> &&
    queryable<O> &&
    is_object_v<O> &&
    requires (O& o) {
      { execution::start(o) } noexcept;
    };
```

In [exec.snd.concepts]/1, edit as follows:

```
template<class Sndr>
  inline constexpr bool ~~enable_sender~~enable-sender = // exposition only
    requires {
      requires derived_from<typename Sndr::sender_concept, sender_t>;
    };

template<is-awaitable<env-promise<empty_env>> Sndr> // [exec.awaitables]
  inline constexpr bool ~~enable_sender~~enable-sender<Sndr> = true;

template<class Sndr>
  concept sender =
    ~~enable_sender~~enable-sender<remove_cvref_t<Sndr>> &&
    requires (const remove_cvref_t<Sndr>& sndr) {
      { execution::get_env(sndr) } -> queryable;
    } &&
    move_constructible<remove_cvref_t<Sndr>> &&  // rvalues are movable, and
    constructible_from<remove_cvref_t<Sndr>, Sndr>; // lvalues are copyable

template<class Sndr, class Env = empty_env>
  concept sender_in =
    sender<Sndr> &&
    requires (Sndr&& sndr, Env&& env) {
      { execution::get_completion_signatures(std::forward<Sndr>(sndr), std::forward<Env>(env)) } ->
        valid-completion-signatures;
    };

template<class Sndr, class Rcvr>
  concept sender_to =
    sender_in<Sndr, env_of_t<Rcvr>> &&
    receiver_of<Rcvr, completion_signatures_of_t<Sndr, env_of_t<Rcvr>>> &&
    requires (Sndr&& sndr, Rcvr&& rcvr) {
      execution::connect(std::forward<Sndr>(sndr), std::forward<Rcvr>(rcvr));
    };
```

Strike [exec.snd.concepts]/3:

~~Remarks: Pursuant to [namespace.std], users can specialize enable_sender to true for cv-unqualified program-defined types that model sender, and false for types that do not. Such specializations shall be usable in constant expressions ([expr.const]) an~~

In [exec.snd.concepts]/6, edit as follows:

Library-provided sender types:

- Always expose an overload of a ~~customization of~~member connect that accepts an rvalue sender.

- Only expose an overload of a ~~customization of~~member connect that accepts an lvalue sender if they model copy_constructible.

- Model copy_constructible if they satisfy copy_constructible.

In [exec.awaitables]/5, edit as follows:

```
  template<class T, class Promise>
    concept has-as-awaitable = requires (T&& t, Promise& p) {
      { std::forward<T>(t).as_awaitable(p) } -> is-awaitable<Promise&>;
    };


  template<~~class~~has-as-awaitable<Derived> T>
    ~~requires tag_invocable<as_awaitable_t, T, Derived&>~~
      auto await_transform(T&& value) noexcept(noexcept(std::forward<T>(value).as_awaitable(declval<Derived&>())))
        ~~noexcept(nothrow_tag_invocable<as_awaitable_t, T, Derived&>)~~
    -> ~~tag_invoke_result_t<as_awaitable_t, T, Derived&>~~ decltype(std::forward<T>(value).as_awaitable(declval<Derived&>())) {
    return ~~tag_invoke(as_awaitable,~~ std::forward<T>(value)~~,~~.as_awaitable(static_cast<Derived&>(*this));
  }
```

In [exec.awaitables]/6, edit as follows:

```
template<class Env>
struct env-promise : with-await-transform<env-promise<Env>> {
  unspecified get_return_object() noexcept;
  unspecified initial_suspend() noexcept;
  unspecified final_suspend() noexcept;
  void unhandled_exception() noexcept;
  void return_void() noexcept;
  coroutine_handle<> unhandled_stopped() noexcept;

  ~~friend const Env& tag_invoke(get_env_t, const env-promise&) noexcept;~~
  const Env& get_env() const noexcept;
};
```

In [exec.getcomplsigs]/1, edit as follows:

execution::get_completion_signatures is a customization point object. Let s be an expression such that decltype((s)) is S, and let e be an expression such that decltype((e)) is E. Then execution::get_completion_signatures(s, e) is expression-equivalent to:

1. ~~tag_invoke_result_t<get_completion_signatures_t, S, E>{}~~decltype(s.get_completion_signatures(e)){} if that expression is well-formed,

2. ...

In [exec.connect]/3, edit as follows:

Let *connect-awaitable-promise* be the following class:

```
struct connect-awaitable-promise : with-await-transform<connect-awaitable-promise> {
  DR& rcvr; // exposition only

  connect-awaitable-promise(DS&, DR& r) noexcept : rcvr(r) {}

  suspend_always initial_suspend() noexcept { return {}; }
  [[noreturn]] suspend_always final_suspend() noexcept { std::terminate(); }
  [[noreturn]] void unhandled_exception() noexcept { std::terminate(); }
  [[noreturn]] void return_void() noexcept { std::terminate(); }

  coroutine_handle<> unhandled_stopped() noexcept {
    execution::set_stopped((DR&&) rcvr);
    return noop_coroutine();
  }

  operation-state-task get_return_object() noexcept {
    return operation-state-task{
      coroutine_handle<connect-awaitable-promise>::from_promise(*this)};
  }

  ~~friend~~ env_of_t<const DR&> ~~tag_invoke~~get_env(~~get_env_t, const connect-awaitable-promise& self~~) const noexcept {
    return execution::get_env(~~self.~~rcvr);
  }
};
```

In [exec.connect]/4, edit as follows:

Let *operation-state-task* be the following class:

```
struct operation-state-task {
  using promise_type = connect-awaitable-promise;
  coroutine_handle<> coro; // exposition only

  explicit operation-state-task(coroutine_handle<> h) noexcept : coro(h) {}
  operation-state-task(operation-state-task&& o) noexcept
    : coro(exchange(o.coro, {})) {}
  ~operation-state-task() { if (coro) coro.destroy(); }

  ~~friend~~ void ~~tag_invoke(start_t,~~ start(~~operation-state-task& self~~) & noexcept {
```

```
      self.coro.resume();
    }
  };
```

In [exec.connect]/6, edit as follows:

> If S does not satisfy sender or if R does not satisfy receiver, execution::connect(s, r) is ill-formed. Otherwise, the expression execution::connect(s, r) is expression-equivalent to:
>
> 1. ~~tag_invoke(connect, s, r)~~s.connect(r) ~~if connectable with tag_invoke<S, R> is modeled~~if that expression is well-formed.
>
>   *Mandates:* The type of the ~~tag_invoke~~ expression above satisfies operation_state.
>
> 2. Otherwise, *connect-awaitable*(s, r) if that expression is well-formed.
>
> 3. Otherwise, execution::connect(s, r) is ill-formed.

In [exec.adapt.general]3,4,5, edit as follows:

> Unless otherwise specified, a sender adaptor is required to not begin executing any functions that would observe or modify any of the arguments of the adaptor before the returned sender is connected with a receiver using execution::connect, and execution::start is called on the resulting operation state. This requirement applies to any function that is selected by the implementation of the sender adaptor.
>
> Unless otherwise specified, a parent sender ([async.ops]) with a single child sender s has an associated attribute object equal to *FWD-QUERIES*(execution::get_env(s)) ([exec.fwd.env]). Unless otherwise specified, a parent sender with more than one child senders has an associated attributes object equal to empty_env{}. These requirements apply to any function that is selected by the implementation of the sender adaptor.
>
> Unless otherwise specified, when a parent sender is connected to a receiver r, any receiver used to connect a child sender has an associated environment equal to *FWD-QUERIES*(execution::get_env(r)). This requirements applies to any sender returned from a function that is selected by the implementation of such sender adaptor.

Strike [exec.adapt.general]6:

> ~~For any sender type, receiver type, operation state type, queryable type, or coroutine promise type that is part of the implementation of any sender adaptor in this subclause and that is a class template, the template arguments do not contribute to the associated entities ([basic.lookup.argdep]) of a function call where a specialization of the class template is an associated entity.~~
>
> ~~[Example:...~~

For the sender adapters, the changes from P2999 regarding how algorithms are customized already removes tag_invokes, so no further changes are needed to them.

Change [exec.as.awaitable]2 as follows:

> as_awaitable is a customization point object. For some subexpressions expr and p where p is an lvalue, Expr names the type decltype((expr)) and Promise names the type decltype((p)), as_awaitable(expr, p) is expression-equivalent to the following:
>
> ~~tag_invoke(as_awaitable, expr, p)~~expr.as_awaitable(p) if that expression is well-formed.
>
> *Mandates: is-awaitable*<A, Promise> is true, where A is the type of the ~~tag_invoke~~ expression above.